

AI Architecture Review System

PRN232-SE1815, Group 3

Đào Nguyên Trọng, Võ Văn Cường (Leader), Hà Huy Hoàng,
Vũ Ngọc Quang, Bùi Phạm Tấn Khang

FPT University, Ho Chi Minh Campus, Vietnam

{trongdnse181776, cuongvvse183671,

hoanghhse180557, quangvnse180410, khangbntse182249}@fpt.edu.vn

Mục lục

1	System Context View	3
1.1	Mô tả phạm vi và tác nhân	3
1.2	Sơ đồ ngữ cảnh (Context Diagram)	3
1.3	Sơ đồ ca sử dụng (Use case Diagram)	4
2	Runtime View	5
2.1	Danh mục dịch vụ (Service Catalog)	5
2.2	Sơ đồ kiến trúc động (Runtime Architecture)	6
2.3	Ma trận giao tiếp (Communication Matrix)	7
2.4	Quy trình tuần tự (Sequence Diagrams)	8
2.4.1	Quy trình Upload Document và phân tích qua AI	8
2.4.2	Quy trình Upload Diagram và Phân tích qua AI	10
2.5	Swagger/OpenAPI Specifications (Đặc tả Swagger)	11
2.5.1	1. UserAuthService Swagger (Port 7158)	11
2.5.2	2. DiagramManagerService Swagger (Port 7065)	11
2.5.3	3. AIArchitectureReviewer Swagger (Port 7257)	13
2.6	gRPC Contract Definition (.proto)	14
2.7	Dịch vụ chạy ngầm (Background Workers)	14
3	Deployment View	16
3.1	Sơ đồ triển khai vật lý (Deployment Diagram)	16
3.2	Giải trình các câu hỏi cốt lõi về hạ tầng	16
3.3	Cấu hình tệp tin điều phối Docker Compose	18

3.4	Minh chứng triển khai hệ thống thực tế (Evidence)	21
-----	---	----

1 System Context View

1.1 Mô tả phạm vi và tác nhân

Hệ thống **AI Architecture Reviewer** là một nền tảng hỗ trợ các kỹ sư phần mềm đánh giá chất lượng thiết kế của các sơ đồ UML (như Class Diagram, Sequence Diagram, ERD...) thông qua trí tuệ nhân tạo.

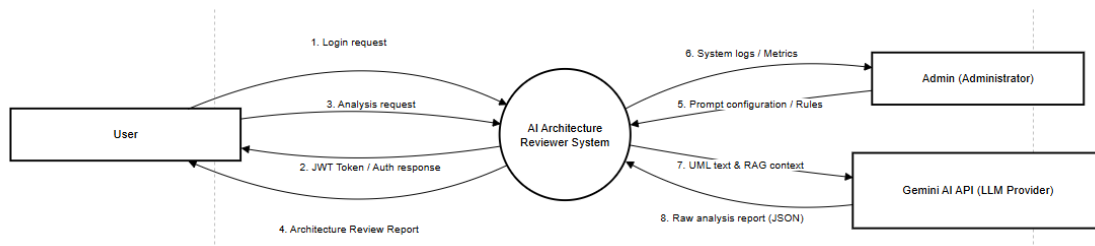
Các tác nhân tương tác (Actors)

- **User (Software Engineer/Architect):** Đăng nhập, quản lý workspace, upload sơ đồ UML, gửi yêu cầu AI phân tích và xem báo cáo phản hồi.
- **Admin:** Quản lý cấu hình Prompt, hệ thống luật thiết kế của AI (Rules), giám sát log và xem báo cáo thống kê.

Danh sách hệ thống bên ngoài (External Systems)

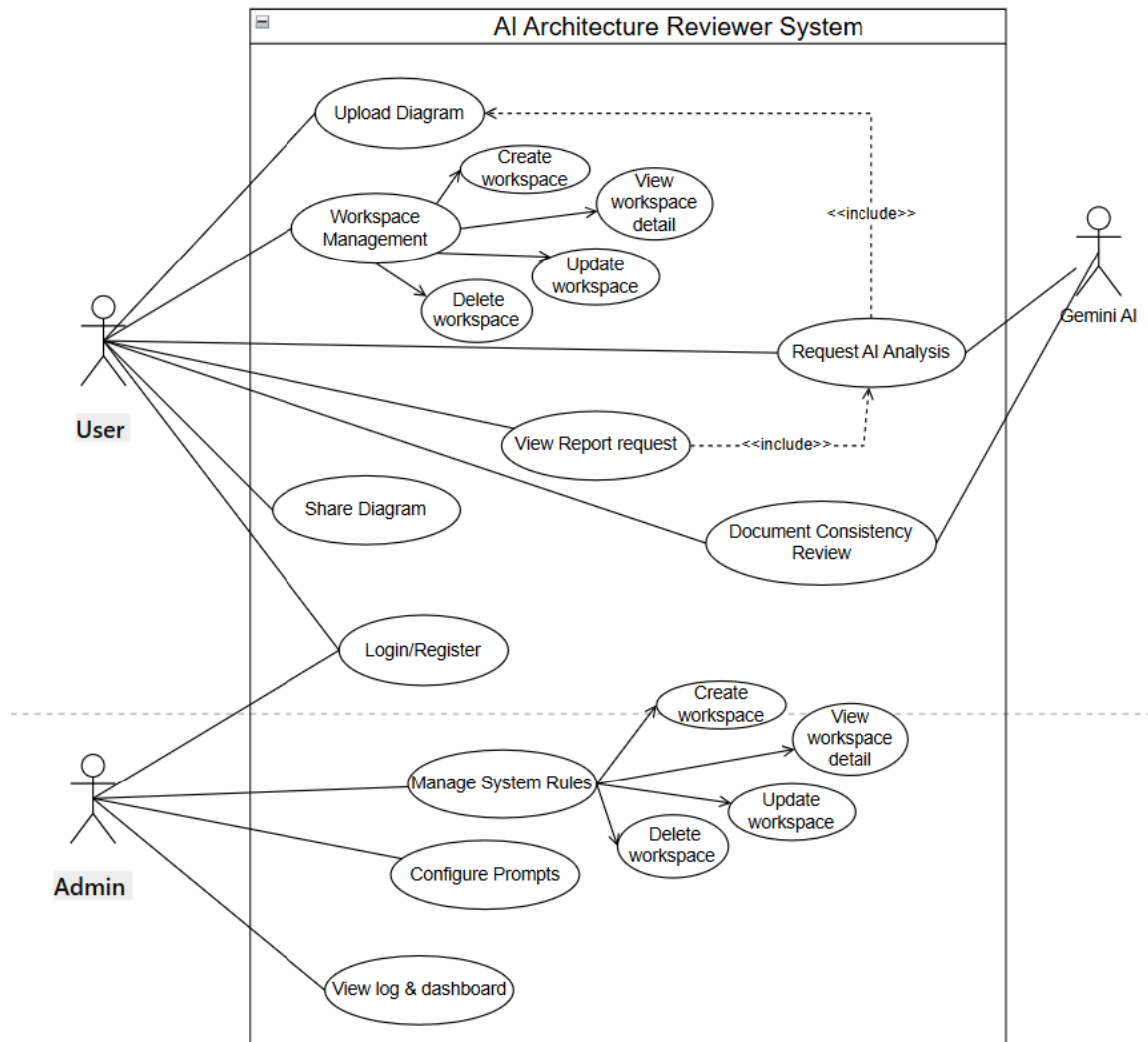
- **Gemini AI API:** Cung cấp mô hình ngôn ngữ lớn để tiếp nhận mã nguồn sơ đồ và trả lời kết quả đánh giá phân tích kiến trúc.

1.2 Sơ đồ ngữ cảnh (Context Diagram)



Hình 1: Sơ đồ ngữ cảnh hệ thống (System Context Diagram)

1.3 Sơ đồ ca sử dụng (Use case Diagram)



Hình 2: Sơ đồ Use Case tổng quan

2 Runtime View

2.1 Danh mục dịch vụ (Service Catalog)

Hệ thống bao gồm tổng cộng **5 dịch vụ** chính hoạt động độc lập (gồm 1 API Gateway, 3 Microservices nghiệp vụ và 1 Message Broker) được đặc tả chi tiết trong Bảng 1.

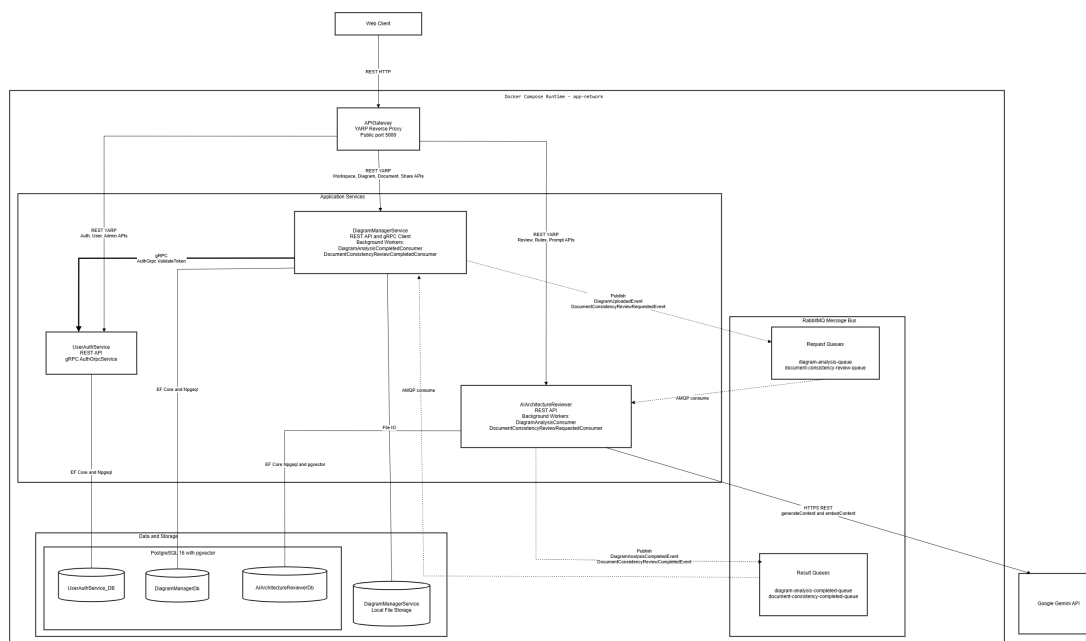
Bảng 1: Danh mục chi tiết các dịch vụ trong hệ thống

Tên Service	Giao tiếp với	Hình thức giao tiếp
UserAuthService	Không chủ động gọi service khác (Chỉ nhận cuộc gọi xác thực gRPC)	gRPC (Server)
DiagramManager	UserAuthService, AIReviewer (gián tiếp qua RabbitMQ)	gRPC (Client), Message Queue
AIReviewer	DiagramManager (gián tiếp qua RabbitMQ), Gemini AI API	Message Queue, REST API (HTTPS)
RabbitMQ	DiagramManager, AIReviewer	AMQP (Message Queue)

Chức năng chính của từng dịch vụ thành phần:

- **UserAuthService:** Quản lý tài khoản người dùng, cấp phát JWT và cung cấp gRPC Server xác thực token tức thời.
- **DiagramManager:** Nghiệp vụ cốt lõi quản lý Workspace, sơ đồ UML, quản lý tài liệu và chia sẻ liên kết.
- **AIReviewer:** Tiếp nhận sơ đồ UML, đối sánh luật bằng kỹ thuật RAG và gọi Gemini API để phân tích lỗi kiến trúc (SOLID, Layer Violation, God Class...).
- **RabbitMQ:** Message Broker trung gian hỗ trợ truyền nhận thông điệp sự kiện bất đồng bộ giữa Diagram Service và AI Service.

2.2 Sơ đồ kiến trúc động (Runtime Architecture)



Hình 3: Sơ đồ kiến trúc động của hệ thống lúc vận hành

2.3 Ma trận giao tiếp (Communication Matrix)

Hệ thống kết hợp linh hoạt 3 cơ chế giao tiếp mạng tùy theo ngữ cảnh nghiệp vụ:

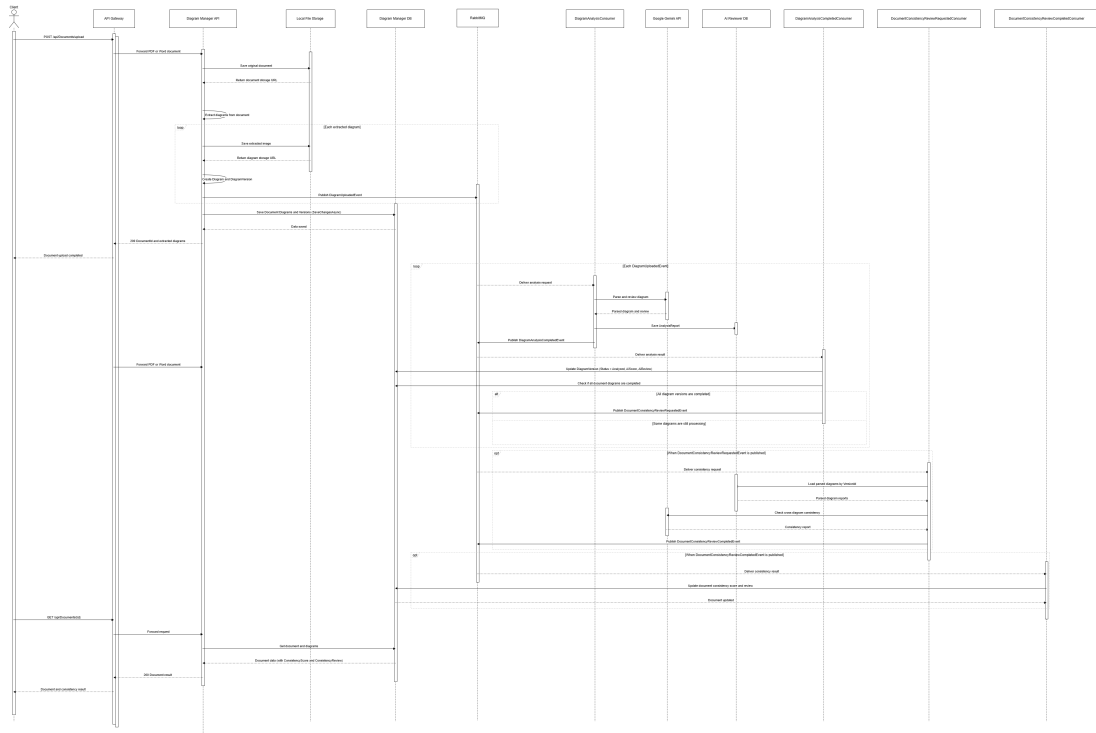
- **HTTP/REST (Gateway/Client):** Sử dụng từ phía Client thông qua API Gateway để thực thi các tác vụ CRUD thông thường.
- **gRPC (Xác thực đồng bộ):** DiagramManagerService gọi trực tiếp UserAuthService qua cổng gRPC nhị phân để validate JWT Token nhanh chóng.
- **AMQP / Message Queue (Bất đồng bộ):** Sử dụng RabbitMQ kết hợp thư viện MassTransit để truyền tải sự kiện mà không gây nghẽn luồng HTTP chính.

Nguồn	Đích	Giao thức	Hình thức	Contract/API/Queue	Mục đích
Frontend Client	APIGateway	HTTP/REST	Đồng bộ	Public API, port 5000	Gửi yêu cầu CRUD, upload và truy vấn dữ liệu
APIGateway	UserAuthService	HTTP/REST qua YARP	Đồng bộ	/api/auth/*, /api/user/*, /api/admin/*	Đăng ký, đăng nhập và quản lý người dùng
APIGateway	DiagramManagerService	HTTP/REST qua YARP	Đồng bộ	/api/Workspaces/*, /api/Diagrams/*, /api/Documents/*, /api/DiagramShares/*	Quản lý workspace, diagram, document và share
APIGateway	AIArchitectureReviewer	HTTP/REST qua YARP	Đồng bộ	/api/Review/*, /api/system-rules/*, /api/prompts/*	Review, chat, consistency, rules và prompts
DiagramManagerService	UserAuthService	gRPC/Protocol Buffers	Đồng bộ	AuthGrpc.ValidateToken	Xác thực JWT và lấy User ID, email, role
DiagramManagerService	AIArchitectureReviewer qua RabbitMQ	AMQP/MassTransit	Bất đồng bộ	DiagramUploadedEvent – diagram-analysis-queue	Yêu cầu AI phân tích một diagram version
AIArchitectureReviewer	DiagramManagerService qua RabbitMQ	AMQP/MassTransit	Bất đồng bộ	DiagramAnalysisCompletedEvent – diagram-analysis-completed-queue	Trả score, review và diagram type
DiagramManagerService	AIArchitectureReviewer qua RabbitMQ	AMQP/MassTransit	Bất đồng bộ	DocumentConsistencyReviewRequestedEvent – document-consistency-review-queue	Yêu cầu kiểm tra consistency của document
AIArchitectureReviewer	DiagramManagerService qua RabbitMQ	AMQP/MassTransit	Bất đồng bộ	DocumentConsistencyReviewCompletedEvent – document-consistency-completed-queue	Trả consistency score và consistency report
AIArchitectureReviewer	Google Gemini API	HTTPS/REST	Đồng bộ	generateContent, embedContent	Phân tích diagram, review, scoring, refactoring và embedding

Hình 4: Ma trận giao tiếp và phân bổ giao thức mạng ảo

2.4 Quy trình tuần tự (Sequence Diagrams)

2.4.1 Quy trình Upload Document và phân tích qua AI



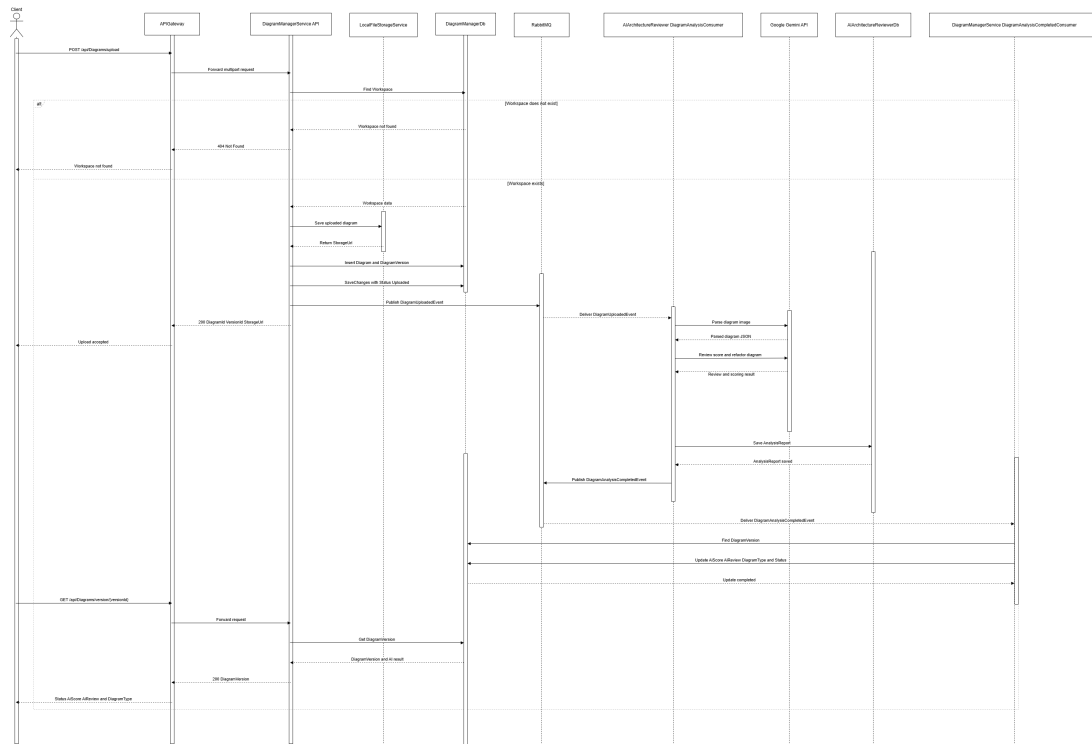
Hình 5: Sơ đồ tuần tự nghiệp vụ tải lên tài liệu và phân tích đối chiếu

Mô tả chi tiết luồng xử lý: Quy trình đối chiếu nhất quán tài liệu yêu cầu (PDF/Word) và sơ đồ diễn ra như sau:

- Bước 1 (Gửi yêu cầu):** Client gửi yêu cầu tải tài liệu kèm ID sơ đồ UML cần đối chiếu lên API Gateway (POST /api/Documents/upload).
- Bước 2 (Định tuyến):** API Gateway chuyển tiếp yêu cầu đến dịch vụ Diagram Manager.
- Bước 3 (Xác thực):** Diagram Manager gọi gRPC sang UserAuthService để kiểm tra tính hợp lệ của token và lấy ID người dùng.
- Bước 4 (Trích xuất sơ đồ):** Diagram Manager lưu file gốc vào File Storage, gọi bộ trích xuất (Document Extractor) để tách các hình ảnh sơ đồ bên trong tài liệu.
- Bước 5 (Đăng ký sự kiện):** Với mỗi sơ đồ trích xuất được, Diagram Manager tạo thực thể sơ đồ mới trong Database và phát sự kiện DocumentConsistencyReviewRequestEvent lên RabbitMQ. Dịch vụ trả về mã 200 OK cho Client.
- Bước 6 (AI phân tích):** AI Analysis Consumer nhận yêu cầu từ RabbitMQ, gọi Gemini API để phân tích sơ đồ và lưu kết quả vào Database của AI Reviewer, sau đó phát sự kiện hoàn thành.

7. **Bước 7 (Đối chiếu nhất quán):** Khi toàn bộ sơ đồ trong tài liệu được phân tích xong, Diagram Result Consumer phát sự kiện đối chiếu chéo. AI Consistency Consumer nhận sự kiện, gọi Gemini đối chiếu văn bản yêu cầu và sơ đồ thiết kế, sau đó lưu kết quả và cập nhật điểm nhất quán cuối cùng.

2.4.2 Quy trình Upload Diagram và Phân tích qua AI



Hình 6: Sơ đồ tuần tự nghiệp vụ tải lên sơ đồ UML trực tiếp

Mô tả chi tiết luồng xử lý: Quy trình tải lên và đánh giá lỗi thiết kế của một sơ đồ UML diễn ra như sau:

- Bước 1:** Client gửi mã nguồn vẽ sơ đồ (Mermaid code) lên API Gateway (POST /api/Diagrams/upload).
- Bước 2:** API Gateway chuyển tiếp yêu cầu và Diagram Manager lưu tệp tin sơ đồ vào File Storage.
- Bước 3:** Diagram Manager ghi thông tin sơ đồ vào cơ sở dữ liệu nội bộ với trạng thái ban đầu là **Pending**.
- Bước 4:** Diagram Manager phát sự kiện **DiagramUploadedEvent** lên RabbitMQ Broker và phản hồi ngay lập tức cho Client trạng thái đang xử lý.
- Bước 5:** Consumer chạy ngầm của dịch vụ AI (**DiagramAnalysisConsumer**) nhận sự kiện, tiến hành chạy tác vụ phân tích RAG kết hợp gọi Google Gemini API dưới nền.
- Bước 6:** Sau khi nhận kết quả chấm điểm từ Gemini, dịch vụ AI lưu báo cáo chi tiết vào DB của AI Reviewer và phát sự kiện **DiagramAnalysisCompletedEvent** lên RabbitMQ.
- Bước 7:** Dịch vụ Diagram Manager nhận sự kiện hoàn thành, cập nhật trạng thái sơ đồ thành **Completed** trong database để Client có thể truy cập xem báo cáo lỗi

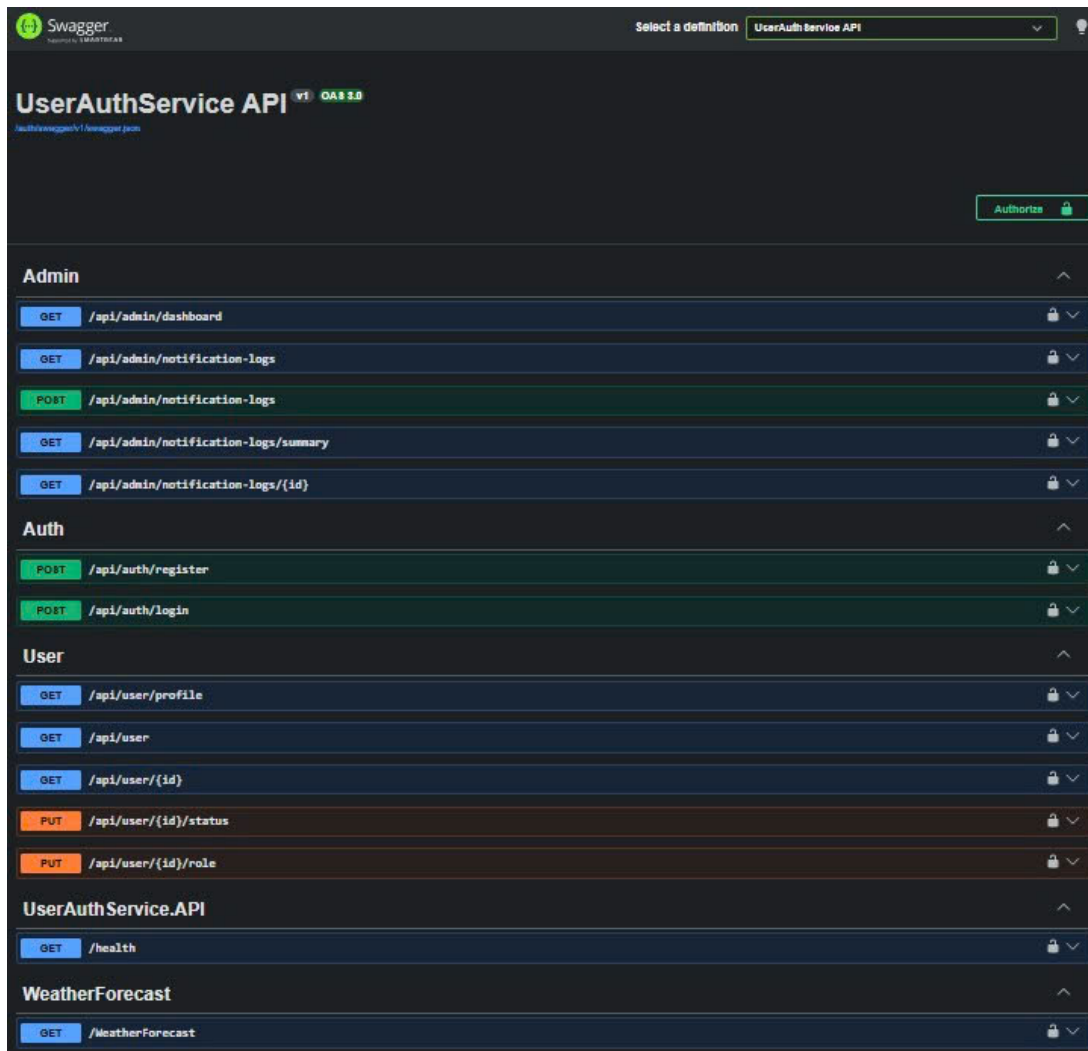
thiết kế.

2.5 Swagger/OpenAPI Specifications (Đặc tả Swagger)

Các Microservices được tích hợp sẵn OpenAPI để tự động sinh tài liệu mô tả đặc tả API:

2.5.1 1. UserAuthService Swagger (Port 7158)

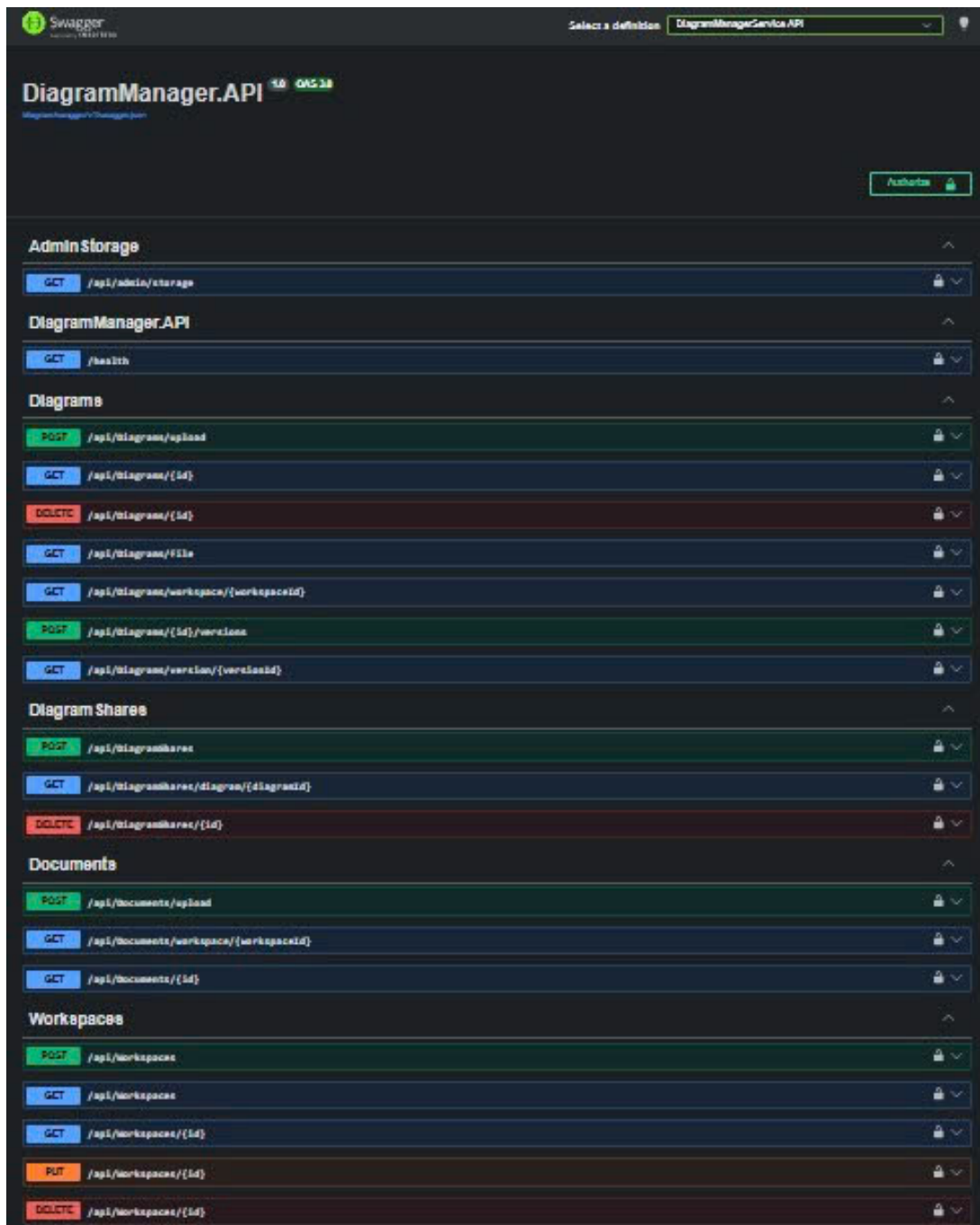
Dịch vụ quản trị thông tin tài khoản, cấu hình phân quyền người dùng.



Hình 7: Giao diện Swagger UI của dịch vụ UserAuthService

2.5.2 2. DiagramManagerService Swagger (Port 7065)

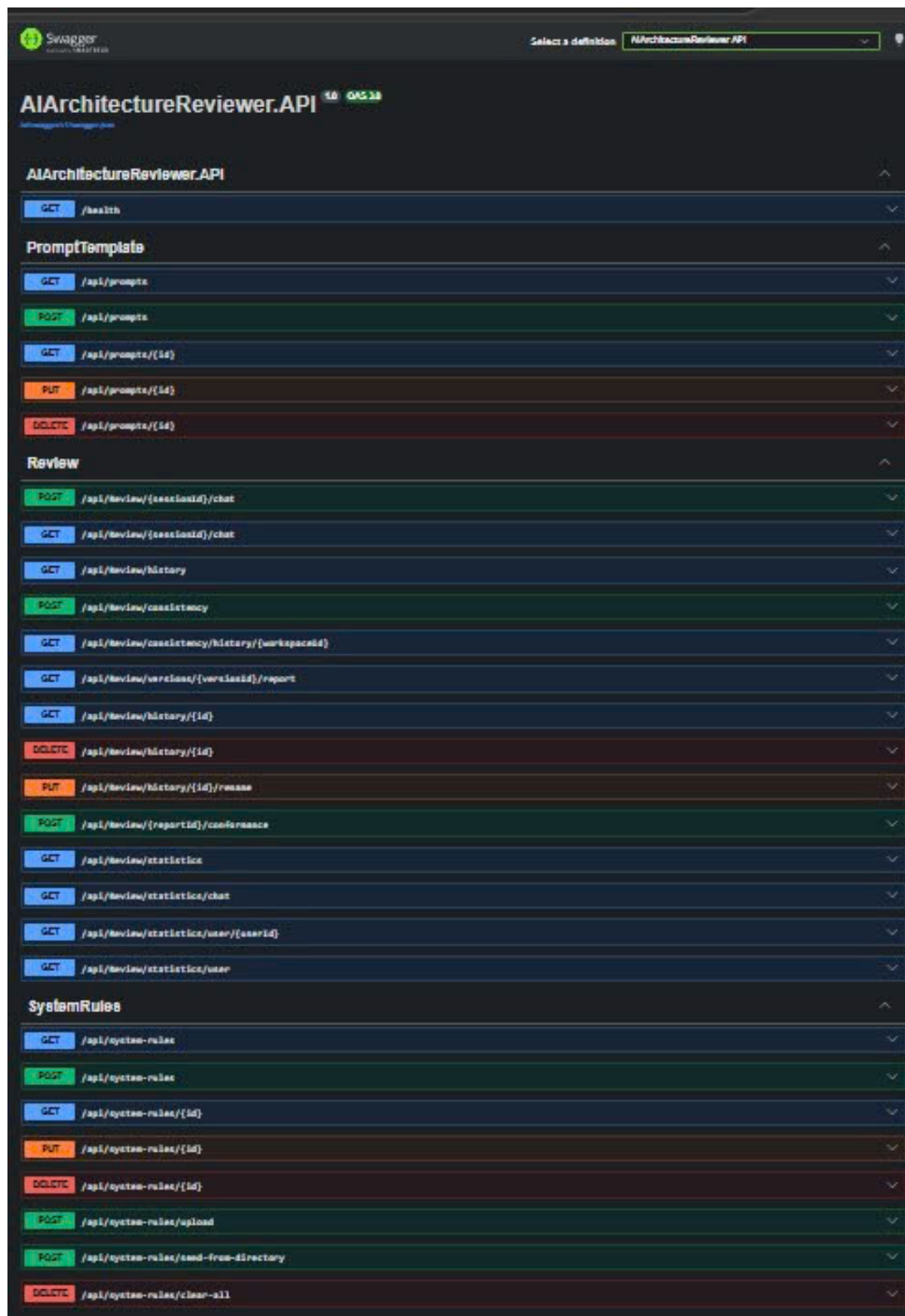
Dịch vụ quản lý vòng đời sơ đồ, tài liệu Word/PDF và các liên kết chia sẻ.



Hình 8: Giao diện Swagger UI của dịch vụ DiagramManagerService

2.5.3 3. AIArchitectureReviewer Swagger (Port 7257)

Dịch vụ cung cấp giao diện quản trị prompt, rules RAG và chat hội thoại sửa lỗi thiết kế.



Hình 9: Giao diện Swagger UI của dịch vụ AIArchitectureReviewer

2.6 gRPC Contract Definition (.proto)

Dưới đây là tệp tin định nghĩa giao thức `auth.proto` dùng để xác thực token nội bộ:

```
syntax = "proto3";

option csharp_namespace = "UserAuthService.API.GrpcServices";

package auth;

service AuthGrpc {
  rpc ValidateToken (TokenRequest) returns (TokenResponse);
}

message TokenRequest {
  string token = 1;
}

message TokenResponse {
  bool isValid = 1;
  string userId = 2;
  string email = 3;
  int32 role = 4;
}
```

Listing 1: Nội dung file contract `auth.proto`

2.7 Dịch vụ chạy ngầm (Background Workers)

Dịch vụ **AIArchitectureReviewer** đóng vai trò là **Background Worker** chính. Nó lắng nghe các hàng đợi sự kiện từ RabbitMQ để phân tích dưới nền mà không gây block request HTTP chính.

Hệ thống triển khai 2 bộ tiêu thụ thông điệp chạy ngầm chính:

1. DiagramAnalysisConsumer:

- *Nhiệm vụ:* Lắng nghe sự kiện tải sơ đồ (`DiagramUploadedEvent`), thực hiện gọi Gemini AI để chấm bài và cập nhật kết quả.
- *File đối chiếu:* `BE/AIArchitectureReviewer/AIArchitectureReviewer.API/Consumers/DiagramAnalysisConsumer.cs`

2. DocumentConsistencyReviewConsumer:

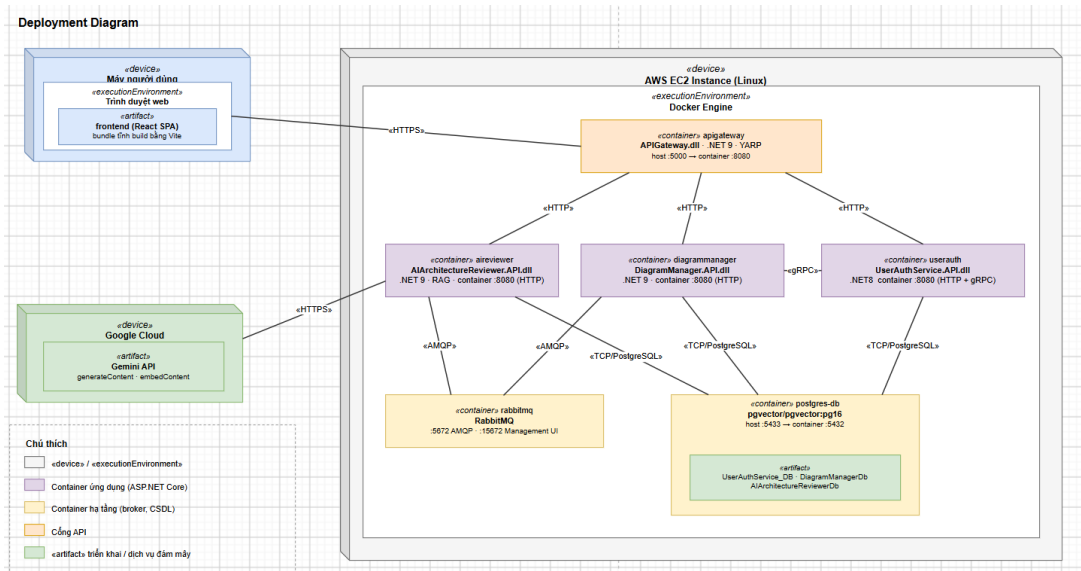
- *Nhiệm vụ:* Lắng nghe sự kiện đối chiếu tài liệu (`DocumentConsistencyReviewRequestedEvent`) so khớp với sơ đồ và trả báo cáo.

- *File đối chiếu:* BE/IArchitectureReviewer/IArchitectureReviewer.API/Consumers/DocumentConsistencyReviewConsumer.cs

3 Deployment View

3.1 Sơ đồ triển khai vật lý (Deployment Diagram)

Kiến trúc phân bố các container vật lý trên Docker Engine, các phân vùng mạng ảo ảo và ổ đĩa lưu trữ được mô tả trong Hình 10.



Hình 10: Sơ đồ phân bố và triển khai vật lý của hệ thống trên Docker Engine

3.2 Giải trình các câu hỏi cốt lõi về hạ tầng

- Số lượng Containers:** Hệ thống chạy tổng cộng **7 Containers** độc lập tương ứng với các dịch vụ nghiệp vụ và hạ tầng hỗ trợ.
- Docker Network:** Tất cả các container kết nối chung thông qua một mạng ảo kiểu Bridge tên là **app-network**, hỗ trợ tự phân giải DNS nội bộ theo tên container.
- Docker Volumes:** Khai báo ổ đĩa ảo **postgres-data** gắn vào đường dẫn CSDL **/var/lib/postgresql/data** của container PostgreSQL nhằm lưu trữ dữ liệu lâu dài (Data Persistence).
- Bảng cấu hình Port Mapping chi tiết:**

Bảng 2: Bảng ánh xạ cổng (Port Mapping) các container

Tên Container	Cổng ánh xạ	Vai trò cổng
web-frontend	3000:80	Cổng truy cập giao diện Web chính của người dùng.
api-gateway	8080:8080	Cổng nhận yêu cầu API từ client để định tuyến nội bộ.
user-auth-service	5001:8080 / 5002:8081	Cổng Web API và cổng gRPC Server chạy ngầm phục vụ xác thực nhanh.
diagram-manager-service	5003:8080	Cổng Web API quản lý sơ đồ và không gian làm việc.
ai-architecture-reviewer	5004:8080	Cổng Web API quản lý Prompt/Rules chạy ngầm.
microservices-db	5433:5432	Cổng kết nối cơ sở dữ liệu PostgreSQL của các microservices.
microservices-rabbitmq	5672:5672 / 15672:15672	Cổng trao đổi sự kiện AMQP và Cổng Web quản lý hàng đợi.

3.3 Cấu hình tệp tin điều phối Docker Compose

Dưới đây là cấu hình hoàn chỉnh của tệp tin `docker-compose.yml` dùng để điều phối và tự động thiết lập toàn bộ hệ sinh thái dịch vụ:

```
version: '3.8'
services:
  microservices-db:
    image: postgres:15-alpine
    container_name: microservices-db
    environment:
      POSTGRES_DB: microservices_db
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: password
    ports:
      - "5433:5432"
    volumes:
      - postgres-data:/var/lib/postgresql/data
    networks:
      - app-network
  microservices-rabbitmq:
    image: rabbitmq:3-management
    container_name: microservices-rabbitmq
    ports:
      - "5672:5672"
      - "15672:15672"
    networks:
      - app-network
  user-auth-service:
    build:
      context: ./BE/UserAuthService
      dockerfile: UserAuthService.API/Dockerfile
    container_name: user-auth-service
    ports:
      - "5001:8080"
      - "5002:8081"
    environment:
      - ConnectionStrings__DefaultConnection=Host=microservices-db;Database=microservices_db;Username=postgres;Password=password
    depends_on:
      - microservices-db
    networks:
      - app-network
  diagram-manager-service:
    build:
      context: ./BE/DiagramManagerService
```

```

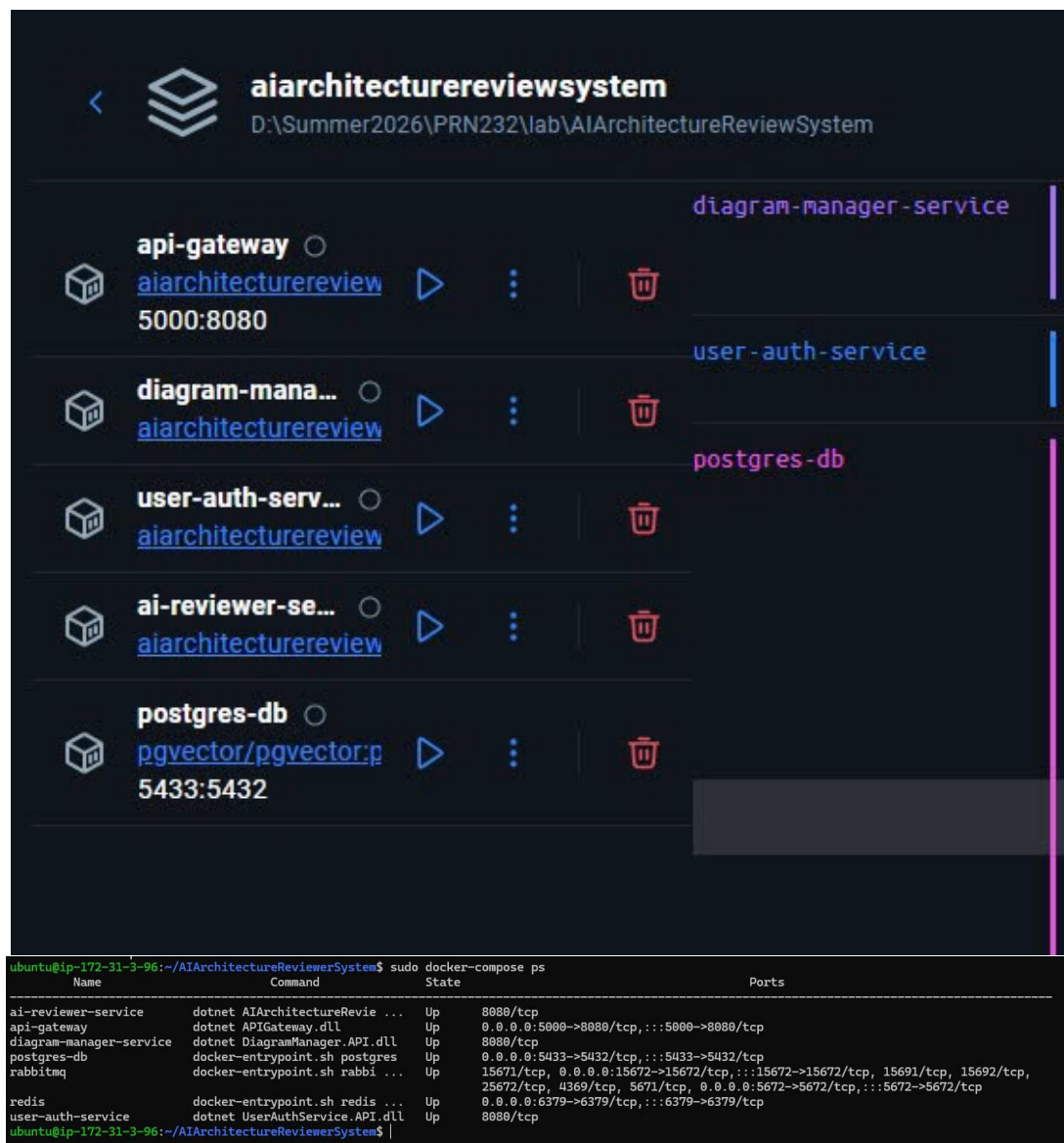
    dockerfile: DiagramManager.API/Dockerfile
  container_name: diagram-manager-service
  ports:
    - "5003:8080"
  environment:
    - ConnectionStrings__DefaultConnection=Host=microservices-db;Database=microservices_db;Username=postgres;Password=password
    - RabbitMQ__Host=microservices-rabbitmq
  depends_on:
    - microservices-db
    - microservices-rabbitmq
  networks:
    - app-network
ai-architecture-reviewer:
  build:
    context: ./BE/AIArchitectureReviewer
    dockerfile: AIArchitectureReviewer.API/Dockerfile
  container_name: ai-architecture-reviewer
  ports:
    - "5004:8080"
  environment:
    - ConnectionStrings__DefaultConnection=Host=microservices-db;Database=microservices_db;Username=postgres;Password=password
    - RabbitMQ__Host=microservices-rabbitmq
  depends_on:
    - microservices-db
    - microservices-rabbitmq
  networks:
    - app-network
api-gateway:
  build:
    context: ./BE/APIGateway
    dockerfile: APIGateway/Dockerfile
  container_name: api-gateway
  ports:
    - "8080:8080"
  depends_on:
    - user-auth-service
    - diagram-manager-service
    - ai-architecture-reviewer
  networks:
    - app-network
web-frontend:
  build:
    context: ./FE/AIArchitectureReviewer-Frontend

```

```
    dockerfile: Dockerfile
  container_name: web-frontend
  ports:
    - "3000:80"
  depends_on:
    - api-gateway
  networks:
    - app-network
volumes:
  postgres-data:
networks:
  app-network:
    driver: bridge
```

Listing 2: Nội dung cấu hình tệp docker-compose.yml

3.4 Minh chứng triển khai hệ thống thực tế (Evidence)



Hình 11: Kết quả khởi chạy thành công toàn bộ các dịch vụ hiển thị qua Docker Desktop và lệnh docker compose ps

— Kết thúc —